PORTFOLIO WEBPROGRAMMIERUNG

LookUpStock

Monte-Carlo-Simulation und fairer Vergleich von Anlagen und Portfolios als Web-Service

LookUpStock ist ein Web-Service, mit dem Privatanleger Aktien und ETFs suchen, daraus gewichtete Portfolios zusammenstellen und deren mögliche Entwicklung mit einer Monte-Carlo-Simulation als Bandbreite darstellen lassen. Zwei Anlagen oder Portfolios lassen sich unter identischen Annahmen vergleichen. Der Service besteht aus zwei Backend-Diensten, einem MPA- und einem SPA-Touchpoint, einer REST- und einer GraphQL-Schnittstelle und bindet den Kursdatenanbieter Twelve Data an.

Team LookUpStock	Rolle im Projekt	GitHub
Mert Eminel	Product Owner	KujoBaraka
Paul Kaufmann	Entwickler	Paul02112004
Peter Orban	Entwickler, Gruppensprecher	Pixxels02
Carl Stibbe	Scrum Master	carlstibbe

Web-Service	https://lookupstockfrontend-96a9998805cb.herokuapp.com
Frontend (MPA, SPA)	https://github.com/LookUpStock/Look-Up-Stock-FrontEnd
Backend (core-service, marktdaten-service)	https://github.com/LookUpStock/Look-Up-Stock-BackEnd
Dokumentation	https://github.com/LookUpStock/Look-Up-Stock-Docs

Duale Hochschule Baden-Württemberg Mosbach · Wirtschaftsinformatik · Kurs WWI25B
Wintersemester 2026/27 · Abgabe am 08.11.2026

Abstract

Dieses Portfolio dokumentiert, wie ein vierköpfiges Studierendenteam in fünf einwöchigen Scrum-Sprints (28.09. bis 01.11.2026) und einem kurzen sechsten Sprint für die Befunde der letzten Abnahme (03.11. bis 09.11.2026) den Web-Service LookUpStock konzipiert und umgesetzt hat. LookUpStock projiziert Anlagen und Portfolios mit einer Monte-Carlo-Simulation und vergleicht sie unter gleichen Annahmen. Im Mittelpunkt stehen nicht die Funktionen, sondern die Entscheidungen und ihre Folgen. Das Team begann mit einem monolithischen Backend und einer Anmeldung über den Request-Body und musste beides umbauen, als die Anforderungen an eine zustandslose, REST-konforme Schnittstelle und an eine nicht monolithische Architektur konsequent angewendet wurden. Die Aufteilung in einen core-service und einen marktdaten-service, die asynchrone Kommunikation über eine Queue-Tabelle in PostgreSQL und die Caches für den Fremdservice Twelve Data entstanden deshalb erst im fünften Sprint. Weitere Lernfelder waren die Grenzen eines kostenlosen Datentarifs, Merge-Konflikte in einem verteilten Team und ein Change Request kurz vor Projektende. Die zentrale Erkenntnis lautet: Die Anforderungen aller Beteiligten früh gegen die Architektur zu prüfen, ist deutlich günstiger als ein späterer Umbau.

Abstract (English)

This portfolio documents how a team of four students designed and implemented the web service LookUpStock in five one-week Scrum sprints (28 September to 1 November 2026) and a short sixth sprint for the findings of the last acceptance test (3 to 9 November 2026). LookUpStock projects assets and portfolios with a Monte Carlo simulation and compares them under identical assumptions. The focus is not on features but on decisions and their consequences. The team started with a monolithic backend and authentication via the request body, and had to rebuild both once the requirements for a stateless, REST-compliant interface and for a non-monolithic architecture were applied consistently. The split into a core service and a market data service, asynchronous communication through a queue table in PostgreSQL, and the caches for the third-party service Twelve Data were therefore only built in the fifth sprint. Further lessons concerned the limits of a free data plan, merge conflicts in a distributed team, and a change request shortly before the end of the project. The central insight is that checking the requirements of all stakeholders against the architecture early on is considerably cheaper than rebuilding it later.

Inhaltsverzeichnis

1 Einführung in das Projekt **4**
1.1 Warum LookUpStock? 4
1.2 Was der Web-Service leistet 4
1.3 Architektur im Überblick 5
1.4 Vorgehen und Verlauf 6
1.5 Aufbau dieses Portfolios 7

2 API-Design **8**
2.1 Vorgehen: Vertrag vor dem Code 8
2.2 Ressourcen und Endpunkte 8
2.3 Zustandslosigkeit: der größte Umbau der Schnittstelle 9
2.4 GraphQL und interne Schnittstellen 10
2.5 Zusammenfassung der Erkenntnisse 10

3 Implementierung **11**
3.2 Integration von Twelve Data, Queue und Caches 14
3.3 MPA und SPA 18

4 Wissenschaftliche Reflexion **21**
4.1 Mert Eminel 21
4.2 Paul Kaufmann 22
4.3 Peter Orban 24
4.4 Carl Stibbe 25

5 Literatur **27**

1 Einführung in das Projekt

Verfasst von Mert Eminel

1.1 Warum LookUpStock?

Wer einen Sparplan auf einen ETF oder ein eigenes Portfolio anlegen möchte, sieht in Vergleichsportalen meist eine einzige Zahl: die Rendite der Vergangenheit. Diese Zahl verdeckt zwei Dinge, die für eine Entscheidung wichtiger sind. Erstens ist die künftige Entwicklung unsicher; ein ungünstiger Verlauf kann deutlich unter den Einzahlungen enden. Zweitens sind Vergleiche oft unfair, weil zwei Anlagen über unterschiedliche Zeiträume oder mit unterschiedlichen Annahmen betrachtet werden.

Die Idee von LookUpStock war deshalb, die Unsicherheit sichtbar zu machen und Vergleiche fair zu gestalten. Eine Monte-Carlo-Simulation rechnet aus der Kurshistorie geschätzte Renditen und Schwankungen in Tausende möglicher Verläufe hoch und zeigt sie als Bandbreite (5., 50. und 95. Perzentil). Beim Vergleich zweier Anlagen oder Portfolios verwenden beide Seiten dieselben Zufallszahlen (Seed) und denselben Schätzzeitraum, sodass Unterschiede nur aus der Zusammensetzung entstehen.

Fachlich bietet das Thema eine echte Rechenlogik statt reiner Datenverwaltung, technisch die Anbindung eines externen Datenanbieters. Bei der Themenwahl war die Machbarkeit in fünf Wochen das wichtigste Kriterium. Als Risiko galt von Anfang an die Simulation selbst. Das Modell (`anforderungen/simulationsmodell.md`) wurde deshalb früh festgelegt und in Sprint 2 bewusst an die bereits programmierte Engine angepasst, statt die Engine neu zu schreiben (normalverteilte Monatsrenditen je Anlage, keine Korrelation). Diese Vereinfachung ist im Modell als Grenze dokumentiert.

Eine Besonderheit bestimmt den gesamten Projektverlauf: Das Projekt wird gleichzeitig in der Fallstudie Software Engineering und in der Webprogrammierung bewertet. Es gab also zwei Anforderungsquellen, die das Team im selben Backlog zusammenführen musste. Wie in Abschnitt 1.4 beschrieben, ist genau an dieser Stelle der größte Fehler des Projekts entstanden.

1.2 Was der Web-Service leistet

Die folgende Übersicht ist bewusst knapp; die Umsetzung beschreibt Kapitel 3.

- **Gäste** probieren auf der Startseite ohne Konto eine Beispielsimulation mit festen Werten aus.
- **Registrierte Nutzer** suchen Aktien und ETFs über Twelve Data, stellen gewichtete Portfolios zusammen, simulieren sie (Fächerdiagramm, Kennzahlen wie Median-Endwert, Value at Risk und Zielwahrscheinlichkeit), vergleichen zwei Einträge und speichern Simulationen.
- **Die Geschäftsführung** des Betreibers sieht monatliche Nutzungskennzahlen in einem Dashboard und exportiert sie als CSV-Datei. Diese Funktion kam als Change Request am 23.10.2026 hinzu (`anforderungen/change-request.md`).

1.3 Architektur im Überblick

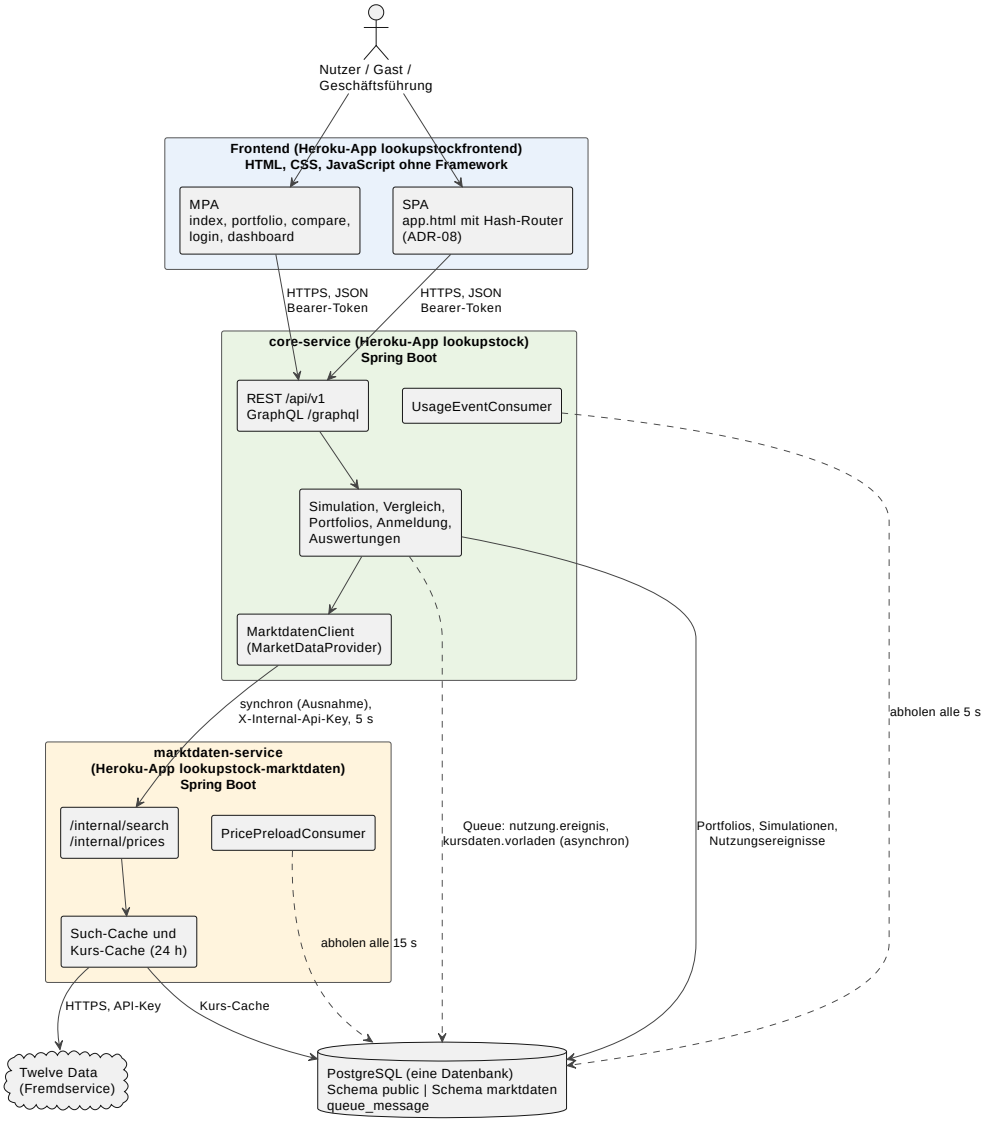


Abbildung 1: Architektur von LookUpStock (Stand Sprint 5). Durchgezogene Pfeile sind synchrone Aufrufe, gestrichelte Pfeile asynchrone Kommunikation über die Queue.

Das Frontend besteht aus HTML, CSS und JavaScript-Modulen ohne Framework und bietet zwei Touchpoints: eine MPA mit fünf Seiten (seit Sprint 5 einschließlich Dashboard) und eine SPA als eigene Einstiegsseite mit Hash-Routing (ADR-08). Beide sprechen ausschließlich mit dem **core-service**, der REST unter `/api/v1` und GraphQL unter `/api/v1/graphql` anbietet und die Fachlogik enthält. Der **marktdaten-service** kapselt den Fremdservice Twelve Data mit Such- und Kurs-Cache und ist nur intern erreichbar. Asynchron laufen das Erfassen der Nutzung und das Vorladen von Kursen über eine Queue-Tabelle in der gemeinsamen PostgreSQL-Datenbank. Die einzige synchrone Verbindung zwischen den Diensten ist die Kursabfrage für die Simulation; sie ist in ADR-07 als begründete Ausnahme festgehalten. Frontend und beide Dienste laufen als eigene Apps auf Heroku.

Die Anforderungen der Guidelines sind wie folgt umgesetzt:

Anforderung der Guidelines	Umsetzung in LookUpStock	Seit	Kapitel
Composable Architecture bzw. MACH (MACH Alliance), kein Monolith	core-service und marktdaten-service mit eigenem Deployment, eigener CI und eigenem Datenbankschema (ADR-07)	Sprint 5	3.1

Anforderung der Guidelines	Umsetzung in LookUpStock	Seit	Kapitel
Kommunikation über eine vorher definierte API, asynchron; begründete Ausnahmen	API-Design als Vertrag vor der Umsetzung; Queue für Nutzungsereignisse und Vorladen der Kurse; synchrone Kursabfrage als Ausnahme	Sprint 1 (API), Sprint 5 (Queue)	2, 3.1, 3.2
Touchpoint nach den Prinzipien einer MPA	fünf Seiten, responsiv ab 360 px (NF-10)	Sprint 1	3.3
Touchpoint nach den Prinzipien einer SPA	`app.html` mit Hash-Router, gleiche Module und Schnittstelle wie die MPA (ADR-08)	Sprint 5	3.3
Persistenz über mindestens ein Backend	PostgreSQL mit Flyway-Migrationen, Schemas `public` und `marktdaten`	Sprint 1	3.1
Daten über REST und GraphQL	REST `/api/v1`; GraphQL lesend (Sprint 4) und schreibend (Sprint 5)	Sprint 1, 4, 5	2, 3.1
Integration eines Fremdservices, mehr als ein GET	Twelve Data: Suche, Monatskurse, Fehlerzuordnung, Cache-Fallback, Vorladen über die Queue	Sprint 1	3.2
Optimierung von Verfügbarkeit und Reaktionsfähigkeit	Kurs-Cache und Such-Cache (24 h), Queue	Sprint 1 bis 5	3.2
Im Internet verfügbar	drei Heroku-Apps	Sprint 2	3.1

1.4 Vorgehen und Verlauf

Das Team arbeitete mit Scrum in fünf einwöchigen Sprints und einem kurzen sechsten Sprint, der nur noch die Befunde der letzten Abnahme behob. Jede Woche entstanden ein Sprint-Planning-Protokoll, Abnahmetestfälle, eine Abnahme durch den Product Owner und ein Sprint Review. Architekturentscheidungen wurden als ADR festgehalten, die Arbeit über GitHub-Issues und Pull Requests in drei Repositories organisiert. Die folgende Tabelle zeigt die Wendepunkte, die das Portfolio in den Kapiteln 2 und 3 aufgreift.

Sprint	Wendepunkt	Folge
1 (28.09.–04.10.)	ADR-01 legt ein einzelnes Backend in drei Schichten fest; getrennte Dienste gelten als zu aufwendig. API-Design als Vertrag vor dem Code.	Schneller Start, aber eine Architektur, die die Guidelines nicht akzeptieren (siehe unten)
2 (05.10.–11.10.)	ADR-04: Token im Request-Body. Deployment auf Heroku. Ein Merge entfernt den Pfad `/api/v1` aus allen Frontend-Aufrufen.	Lesende Abfragen mit Anmeldung laufen als `POST …/list`; erste Erfahrung mit Integrationsfehlern durch Merges
3 (12.10.–18.10.)	Der kostenlose Tarif von Twelve Data liefert keine Xetra-Kurse (Befund F3 aus der Abnahme Sprint 2, PO-Entscheidung vom 18.10.).	Testdaten auf US-Börsen umgestellt; Fehlermeldung „Anlage an einer anderen Börse auswählen“
4 (19.10.–25.10.)	Rückmeldung des Dozenten: Schnittstelle REST-konform. ADR-06: Token im Header, Lesen per GET. GraphQL lesend. Change Request am 23.10. Ein Merge überschreibt vier Fehlerbehebungen.	Umbau der Schnittstelle (Kapitel 2); Branch-Schutz und Review des Diffs in die Definition of Done aufgenommen

Sprint	Wendepunkt	Folge
5 (26.10.–01.11.)	Abgleich mit den Guidelines im Sprint-Planning: Monolith, fehlende Asynchronität und fehlende SPA. ADR-07 und ADR-08.	Aufteilung in zwei Dienste, Queue, SPA und GraphQL-Mutations in einer Woche
6 (03.11.–09.11.)	Die Abnahme von Sprint 5 lag erst nach dem Sprintende und fand zwei Fehler hoher Priorität (B8, B9). Statt die Arbeit außerhalb eines Sprints zu erledigen, beschloss das Team einen kurzen sechsten Sprint ohne neue Funktionen.	Fehlerbehebung mit Nachtest, Branch-Schutz tatsächlich aktiviert, Dokumentation mit dem Code abgeglichen; Lehre: Die Abnahme gehört in den Sprint, nicht danach

Der größte Fehler war ein Prozessfehler. ADR-01 verwarf getrennte Dienste in Sprint 1 aus Aufwandsgründen. Dass die Guidelines eine nicht monolithische Architektur und asynchrone Kommunikation verlangen, wurde erst bei einem vollständigen Abgleich im Sprint-Planning Sprint 5 als Lücke festgehalten (Sprint-Planning-Protokoll Sprint 5, Abschnitt 10). Das Product Backlog war bis dahin vor allem aus den Anforderungen der Fallstudie entstanden. Rückblickend war ADR-01 unter den damals bekannten Annahmen schlüssig, aber die Annahmen waren unvollständig. Für ein künftiges Projekt würde das Team alle Anforderungsquellen schon im ersten Planning in einer gemeinsamen Anforderungsliste zusammenführen und jede Architekturentscheidung dagegen prüfen. Positiv hat sich ausgewirkt, dass das Backend von Beginn an über die Schnittstelle `MarketDataProvider` auf Kursdaten zugriff: Genau an dieser Stelle ließ sich der marktdaten-service in Sprint 5 herauslösen, ohne Kernlogik und Tests umzubauen.

1.5 Aufbau dieses Portfolios

Kapitel 2 beschreibt das API-Design und wie es sich durch die Anforderung der Zustandslosigkeit verändert hat. Kapitel 3, die Implementierung, ist nach Komponenten gegliedert: Abschnitt 3.1 behandelt Architektur, getrennte Dienste und GraphQL, Abschnitt 3.2 die Integration von Twelve Data mit Queue und Caches, Abschnitt 3.3 die beiden Touchpoints MPA und SPA. Innerhalb jedes Abschnitts folgt die Darstellung dem zeitlichen Verlauf über die Sprints. Kapitel 4 enthält die wissenschaftliche Reflexion jedes Teammitglieds, Kapitel 5 die Literatur.

2 API-Design

Verfasst von Mert Eminel

Das vollständige API-Design mit allen Ein- und Ausgaben, Fehlerfällen und Beispielen steht in `api/api-design.md` im Docs-Repository. Dieses Kapitel beschreibt den Aufbau, die Entscheidungen dahinter und was das Team beim Entwerfen und Ändern der Schnittstelle gelernt hat.

2.1 Vorgehen: Vertrag vor dem Code

Das API-Design entstand in Sprint 1 vor der ersten Zeile Backend-Code und lag am 02.10.2026 als erste Fassung vor. Jeder Endpunkt ist in einer Tabelle mit Methode, Ressource, eingehenden Daten (Data In) und ausgehenden Daten mit allen Statuscodes (Data Out) beschrieben. Frontend und Backend konnten so parallel entwickelt werden, weil beide Seiten dieselben Pfade, Felder und Fehlertexte aus dem Dokument kannten.

Der Vertrag war aber ein Textdokument, gegen das nichts automatisch geprüft wurde. Ein Abgleich des Product Owners in Sprint 2 fand mehrere Abweichungen zwischen Dokument und Code, zum Beispiel eine Gültigkeit des Tokens von 7 Tagen statt der festgelegten 24 Stunden und abweichende Fehlertexte. Im selben Sprint entfernte ein Merge den Pfad `/api/v1` aus allen Aufrufen des Frontends. Das Team reagierte organisatorisch: Fachliche Regeln werden als nummerierte Entscheidungen des Product Owners (E1 bis E47) festgehalten und jede Schnittstellenänderung im Dokument mit Datum und Issue vermerkt. In einem künftigen Projekt würde das Team die Schnittstelle zusätzlich maschinenlesbar beschreiben, etwa als OpenAPI-Spezifikation, und Abweichungen mit Vertragstests in der CI finden statt in der manuellen Abnahme.

2.2 Ressourcen und Endpunkte

Die öffentliche Schnittstelle des core-service liegt unter der Basis-URL `/api/v1` und verwendet JSON. Ressourcen sind Substantive im Plural; die Aktion ergibt sich aus der HTTP-Methode.

Ressource	Endpunkte	Anmeldung
Anmeldung	`POST /auth/register`, `POST /auth/login`, `POST /auth/logout`	nur Abmelden
Anlagen	`GET /assets/search?q=&limit=`	nein
Portfolios	`POST /portfolios`, `GET /portfolios`, `GET /portfolios/{id}`, `PUT /portfolios/{id}`, `DELETE /portfolios/{id}`	ja
Simulationen	`POST /simulations` (berechnen; mit `save: true` zusätzlich speichern), `GET /simulations`, `GET /simulations/{id}`, `DELETE /simulations/{id}`	ja
Vergleiche	`POST /comparisons`	ja
Gastmodus	`GET /demo/portfolios`, `POST /demo/simulations`	nein
Auswertungen	`GET /reports/usage?month=`, `GET /reports/usage/export?month=`	ja, Rolle `MANAGER`
GraphQL	`POST /graphql` (unter der Basis-URL, also `/api/v1/graphql`) mit Queries und Mutations	je nach Operation

Eine bewusste Entscheidung betrifft die Berechnungen. Eine Simulation oder ein Vergleich ist kein bloßes Lesen vorhandener Daten, sondern erzeugt ein neues Ergebnis aus umfangreichen Eingaben (Portfolio, Startkapital, Sparrate, Horizont, Pfade, Seed, Zielbetrag). Deshalb laufen beide als `POST` auf eine eigene Ressource. Eine Simulation wird nur gespeichert, wenn der Nutzer das ausdrücklich verlangt; die Antwort ist dann `201 Created` statt `200 OK`. Berechnen und optionales Speichern in einem Endpunkt hielt das Frontend einfach, vermischt aber zwei Zuständigkeiten. Dass das keine reine Stilfrage ist, zeigte die Abnahme Sprint 5: Zum Speichern schickt die Oberfläche dieselbe Anfrage noch einmal mit `save: true`, das Backend rechnete erneut und zählte die Simulation ein zweites Mal als durchgeführt (B8). Seit BE #128 zählt eine Anfrage mit `save: true` nur noch als gespeicherte Simulation; die doppelte Berechnung bleibt. Ein eigener Endpunkt zum Speichern eines berechneten Ergebnisses hätte beides vermieden.

Die Beispielsimulation für Gäste erhielt einen eigenen Pfad `/demo`, statt die Simulation optional ohne Anmeldung zu erlauben. So bleiben die Regeln je Endpunkt eindeutig: `/simulations` verlangt immer eine Anmeldung, `/demo` nie, und für Gäste gilt eine eigene Obergrenze von 10.000 Pfaden.

Statuscodes und Fehler. Alle Fehler haben das Format `{ "error": { "code", "message", "details" } }`. `message` ist ein deutscher Text, den das Frontend unverändert anzeigt; dadurch liegen die Prüfregeln und ihre Meldungen an einer Stelle im Backend. Das Team unterscheidet `400` (ungültige Eingabe), `401` (nicht angemeldet), `403` (angemeldet, aber ohne Berechtigung), `404` (nicht vorhanden oder fremd), `409` (E-Mail bereits registriert), `422` (Eingabe gültig, aber nicht verarbeitbar, z. B. weniger als drei Jahre Kurshistorie) und `503` (Kursdaten nicht verfügbar). Zwei Entscheidungen waren nicht selbstverständlich: Fremde Portfolios liefern `404` statt `403`, damit nicht erkennbar ist, ob eine ID überhaupt existiert (NF-04). Bei den Auswertungen ist es umgekehrt: Wer ohne die Rolle Geschäftsführung zugreift, erhält `403`, weil keine fremde Ressource abgefragt wird, sondern eine Berechtigung fehlt (E40).

2.3 Zustandslosigkeit: der größte Umbau der Schnittstelle

Nach REST enthält jede Anfrage alle Informationen, die der Server zu ihrer Bearbeitung braucht; der Server hält keinen Sitzungszustand zwischen Anfragen (Fielding 2000, Kap. 5.1.3). Die erste Fassung der Anmeldung (ADR-04, Sprint 2) erfüllte das zwar formal, weil jede Anfrage das Token mitschickte, aber sie schickte es im JSON-Body. Da Browser bei `GET` keinen Body senden, mussten lesende Abfragen mit Anmeldung als `POST` auf eigene Pfade ausweichen: `POST /portfolios/list`, `POST /portfolios/{id}/details`, `POST /simulations/list`. Selbst `DELETE` und `POST /auth/logout` brauchten nur wegen des Tokens einen Body. Das Token wurde an 10 Stellen in den Controllern einzeln geprüft.

Die Rückmeldung des Dozenten in Sprint 4 machte deutlich, dass diese Lösung die Semantik von HTTP verletzt: `GET` ist sicher und idempotent, `POST` nicht (RFC 9110, Abschn. 9.2). Mit ADR-06 wurde die Schnittstelle umgebaut:

Vorher (ADR-04)	Nachher (ADR-06)
`POST /portfolios/list` mit `{ "token": "…" }`	`GET /portfolios` mit Header `Authorization: Bearer <token>`
`POST /portfolios/{id}/details`	`GET /portfolios/{id}`
`DELETE /portfolios/{id}` mit Body	`DELETE /portfolios/{id}` ohne Body
Token-Prüfung in jedem Controller	zentrale Auswertung des Headers (`CurrentUserArgumentResolver`, Parameter `@CurrentUser`)

Das Token steht seitdem im Header nach dem Bearer-Schema (RFC 6750); Bodys enthalten nur noch fachliche Daten. Der Umbau kostete in Sprint 4 Backend, Frontend, Testfälle und Diagramme. Die vorhandenen Controller-Tests sicherten jede Änderung ab; die Abnahme Sprint 4 bestätigte den

Umbau (AT-ADR6-01 bis AT-ADR6-05 bestanden).

Objektiv bleibt eine Einschränkung: Das Token ist ein zufälliger Wert, den der Server in der Tabelle `auth_token` nachschlägt. Der Server hält damit Zustand über Anmeldungen, wenn auch keinen Gesprächszustand zwischen Anfragen. Ein signiertes Token wie ein JSON Web Token wäre vollständig zustandslos prüfbar gewesen, hätte aber das sofortige Sperren beim Abmelden (NF-16) erschwert. Das Team hat diesen Zielkonflikt bewusst zugunsten des sofortigen Abmeldens entschieden. Reproduzierbar ist dagegen jede Berechnung: Bei gleichem Seed und gleichen Eingaben liefert `POST /simulations` dasselbe Ergebnis (NF-08), unabhängig davon, welche Anfragen vorher kamen.

Die wichtigste Erkenntnis aus diesem Umbau: Die Anmeldung ist eine Querschnittsentscheidung, die jeden Endpunkt berührt. Sie gehört an den Anfang des API-Designs und nicht in Sprint 2, wenn bereits Endpunkte existieren. Würde das Team neu beginnen, stünde der `Authorization`-Header vom ersten Entwurf an fest.

2.4 GraphQL und interne Schnittstellen

GraphQL bietet über `POST /graphql` vergleichbare Funktionen wie REST: Queries für Anlagensuche, Portfolios und gespeicherte Simulationen (Sprint 4) und Mutations zum Anlegen, Ändern und Löschen von Portfolios, zum Simulieren und zum Vergleichen (Sprint 5). Beide Schnittstellen rufen dieselben Services auf. Prüfregeln, Meldungen und Anmeldung sind deshalb identisch; ein Fehler steht bei GraphQL im Feld `errors` des Ergebnisses (GraphQL Specification 2021, Abschn. 7.1.2), mit demselben Text wie bei REST und dem Code in `extensions.code`; die HTTP-Antwort ist dabei `200 OK`. Die Auswertungen gibt es bewusst nur über REST, weil der CSV-Download ein Dateiformat und keine Graphstruktur liefert.

Mit der Aufteilung in Dienste (ADR-07) kamen zwei interne Schnittstellen hinzu: die Endpunkte des marktdaten-service (`GET /internal/search`, `GET /internal/prices`, geschützt mit dem Header `X-Internal-Api-Key`) und die Themen der Queue (`nutzung.ereignis`, `kursdaten.vorladen`) mit festgelegtem JSON-Inhalt. Beide sind im Abschnitt „Interne Schnittstellen“ des API-Designs beschrieben. Selbstkritisch ist festzuhalten, dass diese Verträge in Sprint 5 parallel zur Umsetzung entstanden und nicht, wie die Guidelines es verlangen, vorher definiert waren. Für die öffentliche Schnittstelle hat das Vorgehen „Vertrag zuerst“ funktioniert; für die internen Schnittstellen fehlte dafür nach der späten Architekturentscheidung die Zeit.

2.5 Zusammenfassung der Erkenntnisse

Entscheidung	Bewertung im Rückblick	Was das Team künftig anders machen würde
API-Design als Dokument vor dem Code	richtig, ermöglichte paralleles Arbeiten	maschinenlesbare Spezifikation und Vertragstests in der CI
Token im Body (ADR-04)	falsch, verletzte die HTTP-Semantik und erzwang einen Umbau	Anmeldung über den `Authorization`-Header vom ersten Entwurf an
Serverseitig gespeichertes Token	vertretbar, bewusster Zielkonflikt mit dem sofortigen Abmelden	Entscheidung früher als ADR dokumentieren
Deutsche Fehlertexte aus dem Backend	richtig, eine Quelle für Regeln und Meldungen	beibehalten
Berechnen und Speichern in einem Endpunkt	funktioniert, aber semantisch unscharf; führte zu doppelter Berechnung und Zählung (B8)	Speichern als eigener Endpunkt, der ein bereits berechnetes Ergebnis übernimmt
Interne Schnittstellen parallel zum Code	erzwungen durch die späte Architekturentscheidung	interne Verträge zusammen mit der Architektur festlegen

3 Implementierung

Die Implementierung ist nach Komponenten gegliedert, so wie das Team die Arbeit im Sprint-Planning Sprint 5 aufgeteilt hat: Abschnitt 3.1 beschreibt Architektur, getrennte Dienste und GraphQL, Abschnitt 3.2 die Integration von Twelve Data mit Caches und Queue, Abschnitt 3.3 die Touchpoints MPA und SPA. Innerhalb jedes Abschnitts folgt die Darstellung dem zeitlichen Verlauf. Jeder Abschnitt ist von den Teammitgliedern geschrieben, die die Komponente umgesetzt haben.

3.1.1 Architektur, getrennte Dienste und GraphQL

Verfasst von Paul Kaufmann

Geplant war LookUpStock als gewöhnliche Spring-Boot-Anwendung mit einem Frontend davor. Nach fünf Sprints besteht das Backend aus zwei Diensten, die sich eine Datenbank teilen und über eine Queue kommunizieren, und GraphQL bietet ungefähr dieselben Funktionen wie REST. Dieser Abschnitt beschreibt den Weg dorthin chronologisch, weil fast jede Entscheidung auf einer früheren aufbaut. Grundlage sind ADR-01 bis ADR-07, die Pull Requests im Backend-Repository und das KI-Log. Twelve Data, Queue und Caches im Detail behandelt Abschnitt 3.2, die Oberflächen Abschnitt 3.3.

3.1.1 Ausgangslage: ein Backend mit drei Schichten

In Sprint 1 haben wir uns in ADR-01 für eine Drei-Schichten-Architektur entschieden: Controller, Services und Repositories in einem einzigen Spring-Boot-Backend. Microservices haben wir bewusst verworfen, weil mehrere Deployments und verteilte Fehlerbehandlung für vier Personen und acht Wochen zu viel schienen. Zwei kleine Entscheidungen aus dieser Zeit haben sich später ausgezahlt: Alle Kursdaten liefen über die Schnittstelle `MarketDataProvider`, ursprünglich nur, damit Tests ohne Internet laufen. Und unsere Use-Case-Controller koordinieren nur, statt selbst zu rechnen.

Der eigentliche Fehler lag woanders. Wir haben ADR-01 nie gegen die Vorgaben der Webprogrammierung geprüft, die monolithische Architekturen ausschließen. Aufgefallen ist uns das erst im Sprint-Planning 5, keine zwei Wochen vor der Abgabe. Heute würden wir die Bewertungskriterien beider Module gleich zu Projektbeginn als Anforderungen aufschreiben.

3.1.2 Vorarbeit: Anmeldung im Header (ADR-06)

Warum das Token mit ADR-06 aus dem JSON-Body in den Header `Authorization: Bearer <token>` wanderte, beschreibt Abschnitt 2.3. Statt den Header in jeder Controller-Methode auszulesen, haben wir einen `HandlerMethodArgumentResolver` geschrieben; ein Controller verlangt einfach `@CurrentUser User user` (gekürzt):

```
@Override
public User resolveArgument(MethodParameter parameter, ModelAndViewContainer mavContainer,
        NativeWebRequest webRequest, WebDataBinderFactory binderFactory) {
    String token = extractToken(webRequest.getHeader(HttpHeaders.AUTHORIZATION));
    if (token == null) {
        throw new UnauthorizedException(Messages.NOT_LOGGED_IN); // 401
    }
    return authService.getObject().authenticate(token);          // 401, wenn abgelaufen
}
```

Damit ist jeder neue Endpunkt automatisch geschützt, und der spätere GraphQL-Interceptor nutzt dieselbe Methode `extractToken`. Weil sich Pfade und Übertragung gleichzeitig änderten, mussten Backend und Frontend im selben Moment gemergt werden. Im Rückblick hätten wir uns den Umweg sparen können: Dass Browser bei GET keinen Body senden, wussten wir schon bei ADR-04, haben aber um das Problem herumgebaut statt es zu lösen.

3.1.3 Aufteilung in zwei Dienste (ADR-07)

Vor dem Umbau haben wir in ADR-07 die Optionen abgewogen. Ein Dienst pro Fachbereich hätte sechs Deployments und verteilte Transaktionen bedeutet. Die Simulation als eigener Dienst klang verlockend, braucht aber Portfolios, Kurse und das Speichern, sodass drei Dienste hätten zusammenarbeiten müssen. Gewählt haben wir zwei Dienste: den core-service mit allem Fachlichen und beiden Schnittstellen für das Frontend, und den marktdaten-service, der als einziger Twelve Data kennt.

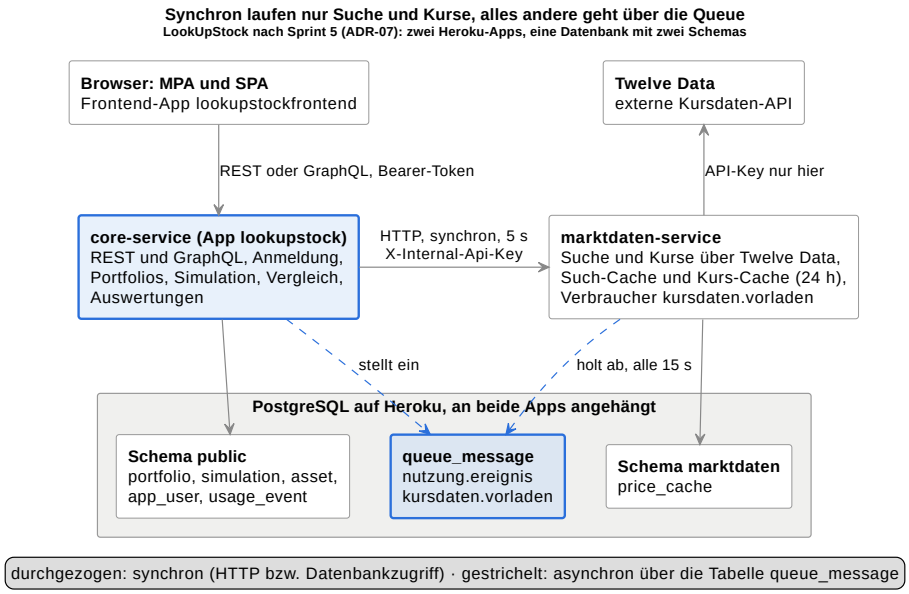


Abbildung 2: Schnitt nach ADR-07: zwei Dienste, eine Datenbank, zwei Schemas

Der Schnitt liegt genau an der Schnittstelle `MarketDataProvider`. Im core-service ersetzt ein `MarktdatenClient` den alten `TwelveDataProvider` und implementiert dieselbe Schnittstelle; für Simulation und Suche änderte sich nichts. Das war der Moment, in dem ein GRASP-Prinzip aus der Vorlesung (Protected Variations) tatsächlich Arbeit gespart hat.

Diesen Aufruf haben wir bewusst synchron gelassen. Die Guidelines erlauben begründete Ausnahmen, und eine Simulation kann erst rechnen, wenn die Kurse da sind; eine Queue hätte nur Wartezeit erzeugt. Der Client wartet höchstens fünf Sekunden, jeder Fehler wird wie früher zu `503`, sodass das Frontend von der Aufteilung nichts merkt. Weil der marktdaten-service als eigene Heroku-App im Internet liegt, lässt ein Filter nur Anfragen mit dem gemeinsamen Schlüssel im Header `X-Internal-Api-Key` durch. Die Datenbank haben wir nicht getrennt: Beide Dienste nutzen eine Datenbank mit zwei Schemas und eigener Flyway-Historie, und der marktdaten-service liest keine Tabelle des Core.

Die synchrone Ausnahme würden wir wieder so wählen, ihr Preis ist aber größer als in ADR-07 beschrieben. Fällt der zweite Dienst aus oder stimmt der Schlüssel nicht, liefern Suche und Simulation `503`, obwohl der Core fehlerfrei ist. Wie viel Zeit der zusätzliche Aufruf kostet, haben wir nicht gemessen.

3.1.4 Umsetzung in kleinen Schritten

Den Umbau haben wir in einzeln mergebare Schritte zerlegt. Zuerst kam ein reiner Verschiebe-PR: Der Code wanderte mit `git mv` nach `core-service/`, 140 von 141 Dateien waren reine Umbenennungen. Den marktdaten-service haben wir zuerst vom Feature-Branch auf Heroku deployt und erst gemergt, als der Health-Check „UP“ meldete; sonst hätten Suche und Simulation online sofort mit 503 geantwortet.

Die größten Probleme lagen nicht im Code. Beim Umzug fiel uns auf, dass in `.env.example` seit Sprint 1 ein echter Twelve-Data-Key stand. Wir haben ihn aus der Datei entfernt, er blieb aber in der Git-Historie, bis er in der Abnahme ersetzt wurde. Auf Heroku kosteten uns Buildpack, Datenbank-Anbindung und ein falsch benannter App-Name Zeit, auch weil nur ein Teammitglied Zugang hatte. Den Code für die Dienste und GraphQL haben wir mit KI-Unterstützung erstellt; die KI konnte ihn nicht selbst kompilieren, deshalb liefen vor jedem Push lokal alle Tests, am Ende 302 im Core und 33 im marktdaten-service.

3.1.5 Asynchronität zwischen den Diensten

Erst beim Abgleich mit den Guidelines haben wir gemerkt, dass die Queue nur Teile des Core verband: Nutzungsereignisse stellt der Core ein und verarbeitet sie selbst, zwischen den Diensten lief alles synchron. Die Lösung stand schon in ADR-07: ein zweites Thema `kursdaten.vorladen`. Legt jemand ein Portfolio an oder ändert es, stellt der Core Symbol und MIC-Code der Anlagen in die Queue. Der marktdaten-service holt alle 15 Sekunden höchstens drei Nachrichten ab und lädt fehlende Kurse in seinen Cache. Die Nachricht enthält bewusst keine IDs aus dem Core:

```
{ "instruments": [ { "symbol": "URTH", "micCode": "ARCX" }, { "symbol": "IEMG", "micCode": "ARCX"
} ] }
```

Dank der Schnittstelle `MessagePublisher` aus der Queue (Abschnitt 3.2.3) war ein neues Thema nur ein neuer Absender und ein neuer Verbraucher. Die Lehre: Eine Vorgabe ist nicht erfüllt, nur weil die passende Technik im Projekt existiert. Hätten wir zu jeder Vorgabe die konkrete Interaktion notiert, die sie nachweist, wäre die Lücke schon beim Planen aufgefallen.

3.1.6 GraphQL mit vergleichbarem Umfang

Seit Sprint 4 war GraphQL nur lesend. In Sprint 5 haben wir sechs Mutations ergänzt, je eine pro schreibendem REST-Endpunkt: Portfolio anlegen, ändern und löschen, simulieren mit optionalem Speichern, gespeicherte Simulation löschen und vergleichen. „Vergleichbar“ haben wir wörtlich genommen: Der `MutationController` bindet die Eingaben an dieselben Klassen wie REST und ruft dieselben Services auf (Stand nach der Behebung von B8):

```
@MutationMapping
public SimulationResponse simulate(@Argument SimulationRequest input,
        @ContextValue(name = AuthTokenInterceptor.TOKEN_KEY, required = false) String token) {
    User user = currentUser(token);                // UNAUTHORIZED wie 401 bei REST
    String name = input.validatedName();           // Name vor dem Rechnen prüfen
    SimulationParameters parameters = input.parameters() == null
            ? null : input.parameters().toSimulationParameters();
    SimulationResult result = input.shouldSave()   // beim Speichern kein zweites SIMULATION_RUN
(B8)
            ? simulationService.simulateForSave(input.portfolioId(), parameters, user)
            : simulationService.simulate(input.portfolioId(), parameters, user);
    if (!input.shouldSave()) {
        return SimulationResponse.from(result);
    }
    SavedSimulation saved = savedSimulationService.save(result, name, user);
    return SimulationResponse.from(saved, savedSimulationService.readResult(saved));
}
```

Dass beide Schnittstellen dieselben Services nutzen, zeigte sich auch beim Fehler B8 aus der Abnahme: Die doppelte Zählung gespeicherter Simulationen steckte in REST und GraphQL zugleich, die Behebung im Service gilt für beide.

Drei Details haben wir bewusst entschieden. Die Eingabefelder sind im Schema optional, damit unsere eigenen Prüfregeln mit deutschen Meldungen greifen statt der englischen Standardmeldung von GraphQL. Der Seed ist als `Float` typisiert, weil `Int` nur 32 Bit kann; das ist bis 2^53 exakt, darüber würde stillschweigend gerundet. Praktisch tritt das nicht auf, weil das System nur Seeds unter

einer Million erzeugt, sauber wäre aber ein eigener Datentyp. Und weil GraphQL auch bei Fehlern mit 200 antwortet, stehen Fehlercode und Einzelfehler im Feld `extensions`. Registrierung, Anmeldung, Gastmodus und Auswertungen gibt es nur über REST; das steht ausdrücklich im API-Design.

3.1.7 Fazit

Die Architektur hat sich in der letzten Woche grundlegend verändert, ohne dass wir Kernlogik oder Tests umbauen mussten. Das lag an früheren Entscheidungen: Schnittstellen wie `MarketDataProvider` und `MessagePublisher`, Controller, die nur koordinieren, und ADRs mit Alternativen vor dem Code. Diese würden wir wieder so treffen, ebenso kleine, einzeln gemergte Umbauschritte. Anders machen würden wir den Anfang: Bewertungskriterien von Beginn an als Anforderungen, Schlüssel nie in Beispieldateien, Branch-Schutz ab Sprint 1 und Heroku-Zugang für alle. Offen bleibt die gemeinsame Datenbank, eine bewusste Kopplung, die bei weiterem Wachstum als Nächstes getrennt würde.

3.2 Integration von Twelve Data, Queue und Caches

Verfasst von Peter Orban und Mert Eminel

Dieser Abschnitt beschreibt, wie LookUpStock den Fremdservice Twelve Data einbindet, mit dessen Grenzen umgeht und die Infrastruktur durch Caches und eine Queue entlastet, mit Fokus auf unsere Entscheidungen und die Fehler, aus denen sie entstanden. Den Schnitt in zwei Dienste beschreibt Abschnitt 3.1.

3.2.1 Twelve Data als Fremdservice

Ausgangslage und Auswahl

Für die Monte-Carlo-Simulation braucht LookUpStock historische Monatskurse jeder Anlage, für den Portfolio-Editor eine Suche nach Aktien und ETFs. Nach einem Anbietervergleich in Sprint 1 (BE #16) hat der Product Owner Twelve Data festgelegt (E7). Genutzt werden die Symbolsuche (`symbol_search`) und die Kursreihe (`time_series` mit `interval=1month`). Über einen einfachen GET-Aufruf hinaus werden Antworten in eigene Datentypen übersetzt, Fehlercodes auf verständliche Meldungen abgebildet und Ergebnisse zwischengespeichert und vorgeladen.

Kapselung hinter einer Schnittstelle

Von Anfang an greift die Fachlogik nur über die Schnittstelle `MarketDataProvider` auf Kursdaten zu (NF-12). So laufen alle Unit-Tests ohne Internet und API-Key mit der Attrappe `MockMarketDataProvider` (BE #53, BE #89), und ein Anbieterwechsel betrifft nur eine Klasse. Der API-Key liegt ausschließlich als Umgebungsvariable auf dem Server (NF-06). In Sprint 5 ermöglichte genau diese Schnittstelle den Umzug in den marktdaten-service (Abschnitt 3.1.3).

Grenzen des Anbieters und was wir daraus gelernt haben

Die meisten Probleme zeigten sich nicht in den Unit-Tests, sondern erst in den Abnahmetests gegen den echten Dienst:

- **Tarif statt Fehler (F3, Sprint 2):** Eine Simulation mit einem Xetra-ETF meldete „weniger als 3 Jahre Kursdaten“, mit einer US-Aktie funktionierte sie. Der kostenlose Tarif liefert keine Xetra-Kurse, und unsere Anbindung unterschied die Fehlerantwort nicht von einer kurzen Kursreihe. Twelve Data meldet solche Fehler meist mit HTTP 200 und einem Fehlercode im JSON-Body. Wir ordnen diese Codes seitdem ausdrücklich ein (gekürzt):

```
// marktdaten-service, TwelveDataError: Einordnung der Fehlercodes von Twelve Data
return switch (code) {
    case 401 -> INVALID_KEY;       // API-Key fehlt oder ist ungültig
    case 403 -> PLAN_RESTRICTED;   // Börse nicht im Tarif, z. B. Xetra
    case 429 -> RATE_LIMIT;        // Kontingent pro Minute bzw. Tag ausgeschöpft
    case 400, 404 -> NOT_FOUND;    // Symbol oder Börse unbekannt
    default -> OTHER;
};

// TwelveDataProvider: dauerhaft keine Kurse → leere Liste, alles andere → 503
if (error == TwelveDataError.NOT_FOUND || error == TwelveDataError.PLAN_RESTRICTED) {
    return List.of();
}
throw unavailable(operation, code, message);
```

Der `ParameterEstimator` im core-service unterscheidet dann eine leere Kursreihe (Hinweis, die Anlage an einer anderen Börse auszuwählen) von einer zu kurzen (weniger als 36 Monatswerte), beides als `422`. Beispielportfolios und Testdaten nutzen seitdem Anlagen an US-Börsen (E45).

- **Reproduzierbarkeit (F4, Sprint 2):** Derselbe Seed lieferte beim ersten Lauf ein anderes Ergebnis als danach. Der erste Lauf rechnete mit ungerundeten Kursen vom Anbieter, die späteren mit den auf sechs Nachkommastellen gerundeten aus dem Cache. Seit dem Fix werden frische Kurse ebenso gerundet (NF-08, siehe 3.2.2).
- **Datenqualität (B5, B6, Sprint 3 und 4):** Der laufende, noch nicht abgeschlossene Monat wird verworfen (E22). Die Suche nach „ETHA“ endete mit `500`, weil die Spalte für den Börsencode zu kurz war (Migration V8).
- **Anfragelimit:** Der kostenlose Tarif erlaubt nur wenige Anfragen pro Minute; jede vermiedene Anfrage hält die Anwendung nutzbar. Das begründet die Caches (3.2.2) und die Drosselung des Vorladens (3.2.3).
- **Bereinigte Kurse (beim Schreiben dieses Kapitels aufgefallen):** Das Simulationsmodell geht von um Splits und Dividenden bereinigten Kursen aus (E20). Unser Aufruf setzt den Parameter `adjust` aber nicht. Laut Twelve Data sind Monatskurse um Splits bereinigt; ob Dividenden eingerechnet werden, bleibt offen (Twelve Data 2026). Ob E20 im Betrieb erfüllt ist, haben wir nicht geprüft.

Erkenntnis: Eine Schnittstelle mit Attrappe macht die Fachlogik testbar, schützt aber nicht vor falschen Annahmen über den echten Dienst. F3 hätten wir schon in Sprint 1 bemerken können: Das API-Design verwendete von Beginn an Xetra-ETFs (EUNL, IS3N), und ein einziger Abruf mit dem kostenlosen Schlüssel hätte die Tarifgrenze gezeigt. Der Anbietervergleich prüfte nur die grundsätzliche Verfügbarkeit, nicht unseren Tarif. Künftig würden wir zuerst Probeabrufe gegen den echten Dienst festhalten: eigene Beispielanlagen, je eine Börse, ein unbekanntes Symbol, Anfragelimit und Kursbereinigung.

3.2.2 Caches

LookUpStock nutzt zwei Caches mit unterschiedlicher Technik, weil die Daten unterschiedlich lange gebraucht werden. Beide liegen seit ADR-07 im marktdaten-service.

	Such-Cache	Kurs-Cache
Inhalt	Treffer je Suchbegriff	Monatskurse je Anlage (Symbol, Börse, Datum)
Speicherort	Arbeitsspeicher (`ConcurrentHashMap`)	PostgreSQL, Schema `marktdaten`, Tabelle `price_cache`
Gültigkeit	24 Stunden	24 Stunden; genau 24 Stunden alt gilt als veraltet
Bei Ausfall von Twelve Data	Treffer jünger als 24 Stunden	Kurse jünger als 24 Stunden, sonst `503`

	Such-Cache	Kurs-Cache
Anforderung	NF-02 (Antwortzeit der Suche)	NF-01 (Simulationsdauer), NF-07 (Ausfallsicherheit)

Such-Cache. In Sprint 2 fehlte er ganz (F5, hohe Priorität). Wir haben ihn bewusst im Arbeitsspeicher umgesetzt, weil Suchergebnisse schnell neu zu beschaffen sind und ein Verlust beim Neustart unkritisch ist; `ConcurrentHashMap`, weil mehrere Anfragen gleichzeitig suchen. In der Abnahme Sprint 3 antworteten 20 Suchen aus dem Cache in höchstens 58 ms (AT-02-08); NF-02 verlangt höchstens 1 s.

Kurs-Cache. Kurse werden für jede Simulation gebraucht und überstehen Neustarts nur in der Datenbank. Der Ablauf im `PriceService` (gekürzt):

```
Optional<CachedPrices> cached = priceCache.load(symbol, micCode);
if (cached.isPresent() && isFresh(cached.get().fetchedAt(), now)) {
    return cached.get().prices();                        // kein Aufruf von Twelve Data
}
List<PricePoint> fresh = marketDataProvider.getHistoricalPrices(symbol, micCode, "1month",
outputSize);
List<PricePoint> sorted = fresh.stream()
        .map(p → new PricePoint(p.date(), p.close().setScale(6, RoundingMode.HALF_UP)))  // F4,
NF-08
        .sorted(Comparator.comparing(PricePoint::date))
        .toList();
priceCache.replace(symbol, micCode, sorted, now);      // Fehler beim Speichern nur protokolliert
return sorted;
```

Die Zeile mit `setScale(6, …)` ist die Lehre aus F4: Frische Kurse werden genauso gerundet wie die in `NUMERIC(19,6)` gespeicherten, sonst hängt das Ergebnis eines Seeds von der Herkunft der Kurse ab. Mit Kursen im Cache erfüllte die Simulation NF-01 in der Abnahme Sprint 3 in 20 von 20 Läufen (1.497 bis 1.602 ms, gefordert unter drei Sekunden).

Objektiv betrachtet bringt der Cache bei einem Ausfall weniger, als wir anfangs dachten. NF-07 verlangt, Kurse jünger als 24 Stunden weiterzuverwenden – genau diese werden aber ohnehin ohne Aufruf von Twelve Data geliefert. Ein eigenes Ausfallverhalten gibt es damit nicht: Ältere Kurse werden nicht verwendet, obwohl sie für eine Simulation über 20 Jahre kaum einen Unterschied machen würden. Eine längere Frist für den Ausfallfall, etwa einige Tage mit Hinweis an den Nutzer, hätten wir schon bei der Formulierung von NF-07 abwägen sollen.

Seit ADR-07 wird der Kurs-Cache über Symbol und Börsencode statt über die Anlagen-ID des Core angesprochen; die alte Tabelle `price_history` entfiel mit V11.

Erkenntnis: Ein Cache ist nicht nur eine Optimierung. Er verändert Ergebnisse (F4) und muss wie Fachlogik getestet werden; die Grenzfälle „genau 24 Stunden alt“ und „Cache veraltet, Anbieter nicht erreichbar“ sind deshalb mit vorgegebener Uhr getestet (`PriceServiceTest`).

3.2.3 Queue und asynchrone Kommunikation

Anforderung und Entscheidung

Die Portfolio-Guidelines verlangen asynchrone Kommunikation zwischen den Komponenten, begründete Ausnahmen sind zugelassen. Gleichzeitig kam mit CR-01 die Erfassung von Nutzungsereignissen für ein Dashboard der Geschäftsführung (US-11), die eine Simulation weder verlangsamen noch scheitern lassen darf.

ADR-07 hält drei Varianten fest: einen Message Broker als Heroku-Add-on, eine Queue-Tabelle in der vorhandenen PostgreSQL-Datenbank und Spring-Events. Spring-Events schieden aus, weil sie innerhalb eines Dienstes bleiben und Nachrichten bei einem Neustart verloren gehen; gegen einen Bro-

ker sprachen Kosten, zusätzliche Zugangsdaten und die Einarbeitung in einer Woche. Wir haben uns für die **Queue-Tabelle** entschieden: kostenlos, neustartfest und mit der Test-Datenbank der CI prüfbar. Die Verzögerung durch periodisches Abfragen ist für Nutzungszahlen unkritisch.

Umsetzung

Die Queue ist eine Tabelle `queue_message` mit Thema, Inhalt als JSON, Zeitpunkt, einem Zähler für Fehlversuche und einer Markierung für endgültig fehlgeschlagene Nachrichten (V10, BE #114). Sie wird über zwei Themen genutzt:

- `nutzung.ereignis`: Nach einer erfolgreichen Aktion, etwa einer Anmeldung oder Simulation, stellt der `UsageEventRecorder` ein Ereignis ein. Ein Verbraucher holt alle fünf Sekunden höchstens 50 Nachrichten ab, überträgt sie in die Tabelle der Nutzungsereignisse und löscht sie.
- `kursdaten.vorladen`: Wird ein Portfolio angelegt oder geändert, lädt der marktdaten-service die Kurse der Anlagen im Hintergrund in seinen Cache (alle 15 Sekunden höchstens drei Nachrichten, wegen des Anfragelimits; siehe 3.1.5).

Drei Details haben sich als wichtig erwiesen:

1. **Keine doppelte Verarbeitung.** Die Verbraucher holen Nachrichten mit `SELECT … FOR UPDATE SKIP LOCKED` (PostgreSQL Documentation, „The Locking Clause“). Laufen mehrere Instanzen, überspringt jede die Zeilen, die eine andere gerade bearbeitet:

   ```
   @Query(value = """
           SELECT * FROM queue_message
           WHERE topic = :topic AND failed = false
           ORDER BY id
           LIMIT :limit
           FOR UPDATE SKIP LOCKED
           """, nativeQuery = true)
   List<QueueMessage> lockNextBatch(@Param("topic") String topic, @Param("limit") int limit);
   ```

2. **Ein Ausfall der Queue darf die Aktion nicht treffen.** Das Einstellen läuft in einer eigenen Transaktion, Fehler werden nur protokolliert:

   ```
   @Transactional(propagation = Propagation.REQUIRES_NEW)
   public void publish(String topic, Object payload) {
       String json = objectMapper.writeValueAsString(payload);
       queueMessageRepository.save(new QueueMessage(topic, json, Instant.now(clock)));
   }
   ```

   Ohne `REQUIRES_NEW` hätte ein Datenbankfehler beim Einstellen auch die Transaktion der eigentlichen Aktion als fehlgeschlagen markiert und etwa das Anlegen eines Portfolios zurückgerollt (Spring Framework Documentation, „Transaction Propagation“).

3. **Zeitpunkt der Aktion statt der Verarbeitung.** Sonst würde eine Anmeldung um 23:59:58 am Monatsende im Folgemonat gezählt. Die Monatsgrenzen rechnet die Auswertung nach deutscher Zeit (E38).

Hinter der Schnittstelle `MessagePublisher` ist die Tabelle austauschbar; ein Wechsel auf einen Broker ändert keine Aufrufer (Protected Variations). Erfasst wird in den Services statt in den Controllern, damit eine Aktion über REST und GraphQL gleich gezählt wird.

Fehler und Erkenntnisse

- **Die Reihenfolge der Schritte war entscheidend.** Erst Ereignistabelle (V9), dann Queue (V10), zuletzt die acht Erfassungsstellen – sonst hätten wir alle acht nachträglich umbauen müssen.
- **Kleine Pull Requests machten Fehler sichtbar.** Der PR mit der Queue (BE #120) war zunächst rot, weil `UsageEventConsumer` ohne Dateiendung `.java` übernommen worden war. Der PR mit V9 (BE #119) versprach das Feld `role` in der Login-Antwort, das erst BE #121 nachlieferte. Beides fiel auf, weil jeder PR nur einen Schritt enthielt; seitdem lesen wir den Diff vor dem Merge gegen die eigene Beschreibung.

- **Grenzen unserer Tests.** Monatsgrenzen sind mit vorgegebener Uhr getestet (`UsageReportServiceTest`). Unsere Unit-Tests laufen aber ohne Datenbank und zeigen nur, dass ein Fehler beim Einstellen nicht weitergeworfen wird (`recordDoesNotThrowWhenQueueFails`, `failingQueueDoesNotBreakTheAction`). Dass `REQUIRES_NEW` die Aktion tatsächlich schützt, haben wir aus der Dokumentation abgeleitet, nicht gegen PostgreSQL nachgewiesen.
- **KI-Unterstützung braucht eine Prüfinstanz.** Teile der Queue wurden mit KI-Unterstützung entworfen (KI-Log). Die eigentliche Prüfinstanz waren Unit-Tests und CI, weil der Entwurf erst dort ausgeführt wurde.
- **Was offen bleibt.** Nach drei Versuchen fehlgeschlagene Nachrichten bleiben unbemerkt in der Tabelle. Die Verarbeitungsdauer im Betrieb und den Nutzen des Vorladens für die erste Simulation haben wir nicht gemessen. Beides würden wir in einem weiteren Sprint sichtbar machen.

Quellen im Projekt: ADR-07, Simulationsmodell, Kurzformat-Tabellen (E7, E20, E22, E38, E45), NFRs (NF-01, NF-02, NF-06, NF-07, NF-08, NF-12), Abnahmeprotokolle Sprint 2 bis 4, Change Request CR-01, Pull Requests BE #119 bis #121, KI-Log.

3.3 MPA und SPA

Verfasst von Carl Stibbe

3.3.1 Ausgangslage und Ziel

Das LookUpStock-Frontend wurde zunächst als Multi-Page Application (MPA) umgesetzt. Die Oberfläche verteilt sich auf die HTML-Seiten `index.html`, `login.html`, `portfolio.html` und `compare.html`; mit dem Dashboard kam in Sprint 5 `dashboard.html` hinzu. Bei einer Navigation zwischen diesen Seiten fordert der Browser jeweils ein neues HTML-Dokument an. Die Seiten verwenden gemeinsame JavaScript-Module für Anmeldung, API-Aufrufe, Simulationen und Diagramme.

Im Sprint 5 kam durch die Portfolio-Guidelines die Anforderung hinzu, neben dem MPA-Touchpoint auch einen Touchpoint nach dem SPA-Prinzip bereitzustellen. Das Sprint-Planning hielt fest, dass die SPA dieselbe REST- oder GraphQL-Schnittstelle wie die MPA nutzen und keine zusätzlichen Backend-Endpunkte benötigen sollte. Der Product Owner bestimmte den Umfang: eigene Portfolios ansehen und simulieren sowie gespeicherte Simulationen ansehen. Die bestehende MPA sollte als eigener Touchpoint erhalten bleiben (Sprint-Planning-Protokoll Sprint 5, Abschnitte 1 und 4; ADR-08). Ziel war also nicht, jede MPA-Funktion ein zweites Mal zu bauen, sondern einen vorführbaren Ablauf, in dem die Ansichten ohne neues HTML-Dokument wechseln.

3.3.2 Entscheidung: eigener Einstieg und Hash-Router

ADR-08 vergleicht zwei Entscheidungen: Wo die SPA liegen soll und wie ihre Routen abgebildet werden. Wir entschieden uns für eine zusätzliche Einstiegsseite `app.html` neben den MPA-Seiten. Ein Umbau von `portfolio.html` hätte die Trennung der Touchpoints verwischt und eine bestehende, getestete MPA-Seite kurz vor der Abgabe verändert. Mit dem separaten Einstieg können beide Varianten unabhängig voneinander aufgerufen und verglichen werden. Zur Auffindbarkeit erhielt die MPA-Navigation einen Link „App (SPA)“. „MPA bleibt erhalten“ bedeutet daher, dass ihre bestehenden Seiten und Funktionen bestehen blieben; die Navigation wurde um diesen Einstieg ergänzt.

Für die Navigation verwendeten wir einen kleinen Router ohne Framework und ohne Build-Schritt. Ein Framework-Router hätte zusätzliche Abhängigkeiten und Einarbeitung bedeutet. Eine History-API-Lösung mit Pfaden wie `/app/portfolios/5` hätte außerdem eine Serverregel erfordert, die direkte Aufrufe und Neuladen auf `app.html` zurückleitet. Das wäre sowohl für den `serve`-Server auf Heroku als auch für die lokale Entwicklung zu konfigurieren gewesen.

Stattdessen verwendet die SPA Hash-Routen wie `app.html#/portfolios/5`. Der Browser sendet den Teil nach `#` nicht als Pfad an den Server. Dadurch funktionieren Direktaufrufe, Lesezeichen sowie Zurück und Vor, ohne dass die Deployment-Konfiguration geändert werden muss. Der Nachteil sind

die sichtbaren `#` in den Adressen. Für diese angemeldete Anwendung ohne Suchmaschinen-Anforderung hielten wir das für vertretbar (ADR-08). Eine Überraschung gab es trotzdem: Der `serve`-Server auf Heroku leitet `app.html` mit `301` auf `/app` um; der Hash bleibt erhalten, die SPA ist im Betrieb also unter `/app#/…` erreichbar. Aufgefallen ist das erst in der Abnahme Sprint 5 (H11).

3.3.3 Aufbau und Navigation

`src/app.html` lädt das Grundgerüst mit Kopfzeile, Navigation und dem Inhaltsbereich `#route-view`. Das Modul `src/js/spa/router.js` übersetzt den Hash beim ersten Laden und bei `hashchange` in einen Routennamen und gegebenenfalls eine ID. `src/js/spa/app.js` rendert anschließend die passende Ansicht im Inhaltsbereich.

Der Router reagiert auf den aktuellen Hash, löst ihn auf und übergibt die Route an den Renderer (gekürzter Ausschnitt aus `src/js/spa/router.js`):

```
const route = resolveRoute(hash);
browserWindow.document.title = `${ROUTE_TITLES[route.name]} – LookUpStock (SPA)`;
await render(route);
```

Damit bleibt die Navigation im selben HTML-Dokument; der Renderer ersetzt den Inhalt des Ansichtsbereichs.

Hash-Route	Ansicht und Aktion	Backend-Aufruf
`#/portfolios`	Eigene Portfolios auflisten	`GET /portfolios`
`#/portfolios/:id`	Portfolio-Positionen ansehen und Simulation starten	`GET /portfolios/{id}`, `POST /simulations`
`#/simulations`	Gespeicherte Simulationen auflisten	`GET /simulations`
`#/simulations/:id`	Gespeicherte Simulation mit Diagramm öffnen	`GET /simulations/{id}`
leerer Hash	Auf `#/portfolios` weiterleiten	–
unbekannte Route	Hinweis mit Rückweg zu den Portfolios	–

Die Navigation besteht aus normalen `a`-Elementen mit Hash-Zielen. Das erhält die erwarteten Browserfunktionen und vermeidet eine eigene Klickbehandlung für jeden Link. Je Ansicht wird der Titel des Browser-Tabs angepasst. Nach dem Rendern setzt der Router den Fokus auf die neue Überschrift. Dadurch ist der Ansichtswechsel auch für Tastaturbedienung erkennbar (NF-09, AT-ADR8-04).

3.3.4 Wiederverwendung statt doppelter Fachlogik

Die SPA nutzt dieselbe REST-Schnittstelle und dieselben geprüften Frontend-Module wie die MPA:

- `api.js` führt die geschützten Anfragen mit dem Bearer-Token aus und liefert API-Fehler als `ApiError`.
- `auth.js` liest die Anmeldung. Ohne gültige Sitzung zeigen die SPA-Ansichten einen Hinweis mit Link zur vorhandenen `login.html`.
- `simulation.js` prüft die Eingaben und fasst das Simulationsergebnis zusammen.
- `chart.js` rendert dasselbe Fächerdiagramm wie in der MPA.
- `saved-simulations.js` formatiert die Angaben zu gespeicherten Simulationen.

Die Eingabeprüfung und die Backend-Aufrufe werden daher nicht für die SPA ein zweites Mal implementiert. Das reduziert Abweichungen zwischen den Touchpoints. Fehler wie eine fehlende Anmeldung, ein nicht gefundenes fremdes Portfolio, ungültige Simulationsparameter oder nicht verfügbare Kursdaten werden über dieselbe API-Schicht verarbeitet. Bei asynchron geladenen Listen verhindert eine Versionsprüfung, dass eine Antwort für eine inzwischen verlassene Ansicht noch gerendert wird (gekürzt aus `src/js/spa/app.js`):

```
const data = await listPortfolios(session.token);
if (!page.isCurrent()) return;
renderPortfolioList(page.section, data?.portfolios ?? []);
```

Die Versionsprüfung schützt die Leseansichten vor Antworten, die nach einem Routenwechsel eintreffen. Beim asynchronen Simulationsaufruf fehlt diese Prüfung. Eine verspätete Antwort wird zwar in den vom DOM getrennten Ergebnisbereich der alten Ansicht geschrieben; insbesondere ein 401 kann aber noch den Sitzungszustand beeinflussen. Das ist eine Grenze gegenüber dem in ADR-08 formulierten Ziel, verspätete Antworten nach einem Ansichtswechsel zu verwerfen.

3.3.5 Funktionsvergleich der Touchpoints

SPA und MPA nutzen dieselben Konten und Endpunkte, bieten aber bewusst nicht denselben Umfang (Stand nach Sprint 5):

Funktion	MPA	SPA
Eigene Portfolios ansehen	Ja	Ja
Portfolio anlegen, bearbeiten und löschen	Ja, im Portfolio-Editor	Nicht vorgesehen
Portfolio simulieren und Fächerdiagramm ansehen	Ja	Ja
Simulationen speichern und verwalten	Ja	Gespeicherte Simulationen ansehen; in der SPA nicht anlegen oder löschen
Zwei Portfolios oder Anlagen vergleichen	Ja	Nicht vorgesehen
Anmeldedaten und geschützte API verwenden	Ja	Ja, über dieselbe Anmeldung und API-Schicht
Navigation zwischen Ansichten	Neue HTML-Seite	Hash-Wechsel innerhalb von `app.html`

Das ist der vom Product Owner festgelegte Ausschnitt, keine vollständige Parität zur MPA; in der Vorführung benennen wir diesen Unterschied ausdrücklich.

3.3.6 Prüfung und Ergebnis

Die Routerlogik ist als eigenes ES-Modul implementiert. `tests/router.test.js` nutzt `node:test` und prüft die Portfolioliste als Standardroute, das Lesen von IDs, die Simulationsrouten sowie unbekannte und ungültige Routen. Damit lässt sich die zentrale Routenauflösung ohne Browser testen.

Die manuellen Abnahmetests prüften zusätzlich, dass nach dem ersten Laden keine weitere Anfrage vom Typ `document` folgt, dass Zurück, Vor, Neuladen und direkte Links funktionieren, dass unbekannte Routen und fehlende Anmeldung behandelt werden und dass der Fokus auf die neue Überschrift wechselt. Alle sechs SPA-Testfälle (AT-ADR8-01 bis AT-ADR8-06) waren in der Abnahme Sprint 5 bestanden. Das belegt die vereinbarten Abläufe, ist aber weder eine Messung der Ladezeit noch ein Nachweis voller Gleichwertigkeit mit der MPA.

3.3.7 Rückblick

Die Entscheidung für eine separate `app.html` erwies sich als zweckmäßig: Wir konnten den SPA-Touchpoint ergänzen, ohne die bestehenden MPA-Abläufe zu ersetzen, und beide Varianten im Abnahmetest nebeneinander betrachten. Hash-Routing passte zum statischen Deployment und ließ sich mit einem kleinen, unabhängig testbaren Modul umsetzen. Der kleine eigene Router passte auch zum engen Zeitrahmen von sechs geplanten Stunden für FE #53.

Der wichtigste Kompromiss war der kleinere Funktionsumfang, gedeckt durch die Entscheidung des Product Owners. Verbessern würden wir die Prüfung: Ein zusätzlicher Abnahmetest sollte während einer laufenden Simulation die Route wechseln und zeigen, dass das verspätete Ergebnis verworfen wird, wie ADR-08 es verlangt.

4 Wissenschaftliche Reflexion

Jedes Teammitglied reflektiert in diesem Kapitel einzeln über die eigene Arbeit: was gelernt wurde, welche Rückschläge es gab, wie mit KI gearbeitet wurde und was in einem künftigen Projekt anders laufen soll. Die Abschnitte sind bewusst aus der jeweiligen Rolle geschrieben und ergänzen sich, statt sich zu wiederholen.

4.1 Mert Eminel

Verfasst von Mert Eminel (Product Owner)

Ich war im Projekt Product Owner und zugleich Kunde. Damit lagen die Anforderungen bei mir: die Kurzformat-Tabellen mit den Akzeptanzkriterien und den PO-Festlegungen E1 bis E47, die Priorisierung im Product Backlog, die Abnahmetests und Abnahmeprotokolle jedes Sprints und der Umgang mit dem Change Request CR-01. Im Frontend habe ich die Anlagensuche, die Anmeldemaske, die Beispielsimulation für Gäste und den CSV-Export umgesetzt und mehrere Befunde aus den Abnahmen behoben. Für mich war es das erste Projekt, in dem ich nicht nur mitgearbeitet, sondern entschieden habe, was gebaut wird und wann etwas fertig ist.

Was ich gelernt habe

Anforderungen müssen prüfbar sein

Meine ersten Akzeptanzkriterien klangen vollständig, waren es aber nicht. In den Abnahmetestfällen von Sprint 1 stand bei der Simulation noch „[OFFEN: Definition von ‚3 Jahre Kurshistorie']“. Erst als ein Testfall geschrieben werden musste, wurde klar, dass niemand wusste, ob drei Kalenderjahre, 36 Monatswerte oder etwas anderes gemeint war. Daraus wurde die Festlegung E8: genau 36 Monatswerte. So sind nach und nach über 40 Festlegungen entstanden, meist mit wörtlichen Meldungstexten und genauen Grenzwerten. Ich habe gelernt, dass ein Kriterium erst dann fertig ist, wenn man daraus einen Testfall mit einem eindeutigen erwarteten Ergebnis ableiten kann.

Die Abnahme ist eine ehrliche Rückmeldung

In Sprint 2 wurden von 16 Testfällen der Kern-Story US-04 nur 7 bestanden, 7 konnten noch gar nicht durchgeführt werden. Das war unangenehm, aber genau richtig: Das Protokoll zeigte dem Team schwarz auf weiß, was fehlte, und in Sprint 3 waren alle 16 bestanden. Ich habe gelernt, eine Story auch dann nicht abzunehmen, wenn sie fast fertig ist, und das sachlich mit dem verletzten Kriterium zu begründen statt mit einem Gefühl.

Entscheiden heißt auch weglassen

Die Ereigniskarte zum Reporting für die Geschäftsführung kam am 23.10.2026, mitten im Projekt. Die vollständige Umsetzung hätte mehr als die Hälfte der Kapazität von Sprint 5 gekostet. Ich habe mich für eine teilweise Umsetzung entschieden: Dashboard und CSV-Export ja, automatischer Monatsbericht nein, dafür wurde der Rest von „Vergleiche speichern“ gestrichen. Entscheidend war ein Gedanke, der mir vorher nicht bewusst war: Nutzungsdaten, die nicht erfasst werden, lassen sich später nicht nachholen. Verschieben wäre deshalb teurer gewesen als eine kleinere Lösung sofort.

Rückschläge

Als Product Owner habe ich Funktionen abgenommen, an denen ich selbst mitgeschrieben hatte. Das ist in einem kleinen Team kaum zu vermeiden, macht aber blind für die eigenen Annahmen. Deutlich wurde mir das beim Bedienbarkeitstest NF-09, den Carl durchgeführt hat: Zwei von drei Personen wussten nach dem Speichern eines Portfolios nicht, wie sie zur Simulation kommen. Diesen Weg hatte keine meiner Anforderungen beschrieben, weil er für mich selbstverständlich war.

Der zweite Rückschlag betrifft die Dokumentation. In der Abschlussprüfung von Sprint 6 zeigte sich, dass einige Dokumente den Code nicht mehr abbildeten: Diagramme zeigten noch einen einzigen Dienst, Testklassen trugen ihre geplanten statt ihrer echten Namen, und Fehlertexte im API-Design wichen vom Code ab. Keine dieser Abweichungen war falsch gemeint; sie sind entstanden, weil Dokumente in einem eigenen Pull Request nachgezogen wurden statt im selben wie der Code.

Arbeiten mit KI

Ich habe KI vor allem für Entwürfe genutzt: Testfälle, Protokolle, Kurzformat-Tabellen und Teile des Frontend-Codes. Das hat viel Schreibarbeit gespart. Gleichzeitig formuliert die KI Meldungstexte und Namen, die plausibel klingen, aber nicht dem Code entsprechen. Ich habe mir deshalb angewöhnt, jede Angabe im Dokument gegen den Code zu prüfen. Die Abnahmen selbst habe ich immer selbst im Browser durchgeführt, und jede Entscheidung als Product Owner ist meine geblieben.

Was gut funktioniert hat

- Die PO-Festlegungen mit Nummern machten Entscheidungen nachvollziehbar; im Code, in Tests und in Diskussionen konnte man sich auf „E37“ beziehen statt auf eine Erinnerung.
- Die Abnahmeprotokolle haben den Fortschritt messbar gemacht und Fehler mit Priorität direkt in Issues übersetzt.
- Der Change Request wurde nach einem festen Schema bewertet, mit Optionen, Aufwand und Begründung, statt aus dem Bauch heraus.

Was ich beim nächsten Mal anders machen würde

1. Akzeptanzkriterien direkt mit Grenzwerten und Meldungstexten formulieren, nicht erst beim Schreiben der Tests.
2. Früh mit echten Nutzern testen, nicht erst kurz vor Schluss.
3. Dokumentation im selben Pull Request wie den Code ändern.

Fazit

Die Rolle des Product Owners war für mich anfangs vor allem Verantwortung für Dokumente. Am Ende habe ich sie als Verantwortung für Klarheit verstanden: Je genauer eine Anforderung formuliert war, desto weniger Rückfragen gab es und desto schneller ging die Abnahme. Dass aus einer Idee zu Monte-Carlo-Simulationen ein laufender Web-Service mit zwei Diensten, MPA und SPA und einem Dashboard für die Geschäftsführung geworden ist, ist für mich das schönste Ergebnis. Ich nehme aus dem Projekt mit, wie viel ein Team erreicht, wenn jeder sich auf klare Absprachen verlassen kann.

4.2 Paul Kaufmann

Verfasst von Paul Kaufmann (Entwickler)

Meine Aufgaben im Projekt lagen vor allem im Backend. Ich habe das Portfolio-Backend, die Anlagensuche über Twelve Data, die Anmeldung und die Zuordnung der Portfolios zu ihren Nutzern umgesetzt, später den SimulationService mit Kursdaten-Cache, den Vergleich und das Ändern und Löschen von Portfolios. In Sprint 4 und 5 kamen die Architekturthemen dazu: die Umstellung auf den Authorization-Header (ADR-06), die Aufteilung in core-service und marktdaten-service (ADR-07) mit der zweiten Heroku-App, die schreibenden GraphQL-Mutations und das asynchrone Vorladen der Kurse. Daneben habe ich Diagramme, Sprint-Planning-Protokolle und mehrere ADRs geschrieben.

Was ich gelernt habe

Ich kam mit Erfahrung in Java und C# ins Projekt, aber ohne Spring Boot, Flyway, Heroku oder GraphQL. Der größte Lernschritt war für mich deshalb nicht eine einzelne Technik, sondern das Zusammenspiel. Am Anfang habe ich Tabellen noch von Hand in pgAdmin angelegt und erst auf einen

Hinweis hin als Flyway-Migration nachgezogen. Am Ende habe ich die Reihenfolge von Migrationen aus verschiedenen Issues geplant, damit Heroku beim Start nicht abbricht. Ähnlich ging es mir mit REST: In ADR-04 habe ich lesende Abfragen als POST umgesetzt, weil ein GET keinen Body tragen kann, statt das Token einfach in den Header zu legen. Erst die Rückmeldung des Dozenten hat mir gezeigt, dass ich ein Symptom umgangen hatte, statt das Problem zu lösen. ADR-06 hat das in Sprint 4 korrigiert, kostete aber eine gleichzeitige Umstellung von Backend und Frontend.

Beim Aufteilen in zwei Dienste habe ich verstanden, dass die eigentliche Arbeit nicht im Code liegt, sondern in der Reihenfolge. Hätten wir den marktdaten-service gemergt, bevor die zweite App auf Heroku lief, wären Suche und Simulation sofort ausgefallen. Dass der Umbau gelang, ohne die Simulation anzufassen, lag an einer Entscheidung aus Sprint 1: Die Schnittstelle MarketDataProvider hatten wir nur für Tests ohne Internet eingeführt. Erst hier habe ich wirklich gesehen, was Protected Variations aus der Vorlesung praktisch bedeutet.

Rückschläge

Der deutlichste Rückschlag war Sprint 1: Von 15 Story Points wurde keiner abgenommen. Wir hatten unterschätzt, wie viel Einarbeitung jede neue Technik kostet, und Tasks wie die Koordination der Simulation fehlten im Backlog ganz. Für mich persönlich war die zweite Lehre, dass ich Architekturentscheidungen früher gegen alle Vorgaben hätte prüfen müssen. ADR-01 widersprach den Guidelines der Webprogrammierung, die keinen Monolithen erlauben, und das ist uns erst im Sprint-Planning 5 aufgefallen. Ähnlich war es bei der asynchronen Kommunikation: Die Queue gab es, sie verband aber nicht die beiden Dienste, und auch diese Lücke haben wir erst kurz vor Schluss geschlossen.

Ein Fehler, aus dem ich besonders viel mitnehme, ist der API-Key in der Datei .env.example. Ich habe ihn beim Umzug entfernt, aber unterschätzt, dass er damit in der Git-Historie bleibt. Erst in der Abnahme wurde er ersetzt. Heute würde ich einen Schlüssel, der einmal eingecheckt war, immer sofort neu erzeugen. Auch der Fehler B8 aus der letzten Abnahme hat mir etwas gezeigt: Weil REST und GraphQL dieselben Services nutzen, steckte die doppelte Zählung in beiden Schnittstellen zugleich. Derselbe Grund hat aber auch dafür gesorgt, dass eine einzige Änderung im Service beide repariert hat.

Arbeiten mit KI

Einen großen Teil meines Codes habe ich mit KI-Unterstützung erstellt, oft Schritt für Schritt mit Erklärungen, die ich dann selbst umgesetzt habe. Das hat mir den Einstieg in Spring Boot deutlich erleichtert. Gleichzeitig habe ich gelernt, dass ich die Ergebnisse nicht ungeprüft übernehmen darf. Die KI hat in einem Entwurf eine Anforderung erfunden, im ersten ADR-Entwurf den GRASP-Controller falsch gesetzt und konnte den Code für die Dienste in ihrer Umgebung nicht kompilieren. Meine wichtigste Absicherung waren deshalb die Tests: Vor jedem Push liefen lokal alle, am Ende 302 im core-service und 33 im marktdaten-service. Ohne sie hätte ich einen Umbau dieser Größe kurz vor der Abgabe nicht gewagt.

Bewertung des Umfangs

Rückblickend war der Umfang stellenweise zu groß. Neben den Anforderungen der Fallstudie kamen für die Webprogrammierung getrennte Dienste, asynchrone Kommunikation, GraphQL und ein SPA-Touchpoint dazu, und fast jedes Thema bedeutete, sich erst in eine neue Technik einzulesen. In einwöchigen Sprints neben den Vorlesungen blieb dafür wenig Puffer; am deutlichsten war das in Sprint 5, für den wir rund 20 Stunden pro Person eingeplant hatten. Trotzdem bin ich insgesamt gut zurechtgekommen. Geholfen haben mir vor allem kleine, einzeln gemergte Schritte, die Tests als Sicherheitsnetz und die ADRs, in denen ich Entscheidungen vor dem Code durchdacht habe. Für ein nächstes Projekt würde ich die Bewertungskriterien gleich zu Beginn als Anforderungen aufschreiben, Einarbeitungszeit ausdrücklich einplanen und neue Techniken lieber nacheinander als gleichzeitig einführen.

4.3 Peter Orban

Verfasst von Peter Orban (Entwickler, Gruppensprecher)

Im Rahmen des Webprogrammierungsprojekts an der DHBW haben wir als Gruppe die Webanwendung LookUpStock entwickelt. Mit einer Monte-Carlo-Simulation werden mögliche Aktienkursverläufe berechnet und für Nutzer verständlich dargestellt. Als Entwickler waren unter anderem der Simulationsalgorithmus, Frontendgerüst und Backendgerüst in meinem Aufgabenbereich. Für mich war es das erste Projekt mit Scrum und meine ersten Berührungspunkte mit GitHub.

Schwierigkeiten mit Scrum

User Stories richtig schneiden

Am Anfang waren unsere User Stories zu groß und zu ungenau. Erst mit der Zeit habe ich gelernt, Stories in kleinere, testbare Teile zu zerlegen, sodass auch mehrere Teammitglieder an derselben Story arbeiten konnten.

Aufwand schätzen

Den Aufwand habe ich anfangs oft unterschätzt, vor allem bei technischen Grundlagen wie dem Projekt-Setup (Maven) und der Anbindung von Frontend und Backend. Bei der Planung wurde für das Verstehen der Komponenten zu wenig Zeit eingeplant. Am Sprintende waren deshalb Stories noch offen, und der Plan für den nächsten Sprint geriet durcheinander.

Definition of Done und Abnahme

Wir haben zu spät festgelegt, wann eine Story als erledigt gilt. Bei der Abnahme durch ein Teammitglied hat sich gezeigt, dass jeder etwas anderes unter „fertig“ verstanden hat: Code geschrieben, getestet, dokumentiert oder schon gemergt. Eine klare Definition of Done von Anfang an hätte viele Rückfragen gespart.

Abhängigkeiten im Team

Durch die Einteilung der Issues kam es oft zu Abhängigkeiten der Issues, sodass einige von diesen erst bearbeitet werden konnten, nachdem ein anderes abgeschlossen war. Dabei kam es oft zu Stockungen, wie zum Beispiel bei dem Grundgerüst des Front- und Backends, bei welchem ich mehr Zeit benötigt habe.

Scrum neben Studium und Arbeit

Regelmäßige Meetings und gleichmäßiges Arbeiten über den Sprint waren schwierig. Oft wurde kurz vor dem Sprint Review viel auf einmal erledigt, und das widerspricht eigentlich der Idee von Scrum.

GitHub als Werkzeug

Issues, das Projekt-Board, Branches und Pull Requests waren für mich teilweise neu. Am Anfang war das Board nicht immer aktuell, und so war der echte Projektstand schwer zu erkennen.

Was gut funktioniert hat

- Durch die Sprint Reviews hatten wir regelmäßig ein lauffähiges Zwischenergebnis.
- In Retrospektiven konnten wir Probleme offen ansprechen und die Arbeitsweise anpassen.
- Wir haben früh festgelegt, dass Simulationen nicht gespeichert werden, sondern nur Portfolios pro Nutzer, was spätere Diskussionen vermeiden konnte.
- Die getrennten Repositories für Frontend, Backend und Doku haben für Übersicht gesorgt.

Was ich gelernt habe

- Kleine, klar formulierte User Stories mit Akzeptanzkriterien sind die Grundlage für funktionierende Sprints.
- Probleme früh ansprechen ist wichtiger, als alles allein zu lösen.

- Scrum funktioniert nur, wenn das Board gepflegt wird und alle es ernst nehmen.

Was ich beim nächsten Mal anders machen würde

1. Definition of Done gleich im ersten Sprint festlegen.
2. Stories gemeinsam schätzen und Puffer einplanen.
3. Gleichmäßige Arbeitseinteilung während der Sprints und nicht den Großteil vor dem Review.

Fazit

Große Hürden wie Abstimmung, Kommunikation und eine schwache Schätzung der Issues hätte man mit einer besser durchdachten Planung vermeiden können. Diese Hürden waren jedoch eine Lernerfahrung, um Scrum wirklich zu verstehen. Erst durch diese Probleme wurde mir klar, wie wichtig eine sorgfältige Sprintplanung, klar formulierte User Stories und eine gemeinsame Definition of Done sind. Theoretisch kannte ich diese Elemente schon vorher, ihren praktischen Wert habe ich aber erst im Projekt erkannt.

4.4 Carl Stibbe

Verfasst von Carl Stibbe (Scrum Master)

Meine Aufgaben lagen vor allem im Frontend und bei der Bereitstellung der Anwendung. Ich habe an der SPA mit clientseitigem Routing (FE #53), am Dashboard mit Monatsauswahl und rollenabhängiger Navigation (FE #51) sowie an Responsive-Fixes gearbeitet. In Sprint 6 kamen Fehlerbehebungen an der Navigation und am Dashboard dazu (FE #67 und FE #70). Außerdem habe ich die Heroku-Apps konfiguriert und den übrigen Bereitstellungsaufbau weitgehend allein betreut. Die Einrichtung der Datenbank hat Paul übernommen. Zu meinen Aufgaben gehörten unter anderem die getrennten Apps für core-service und marktdaten-service sowie deren Konfiguration. Als Scrum Master habe ich zusätzlich auf die Abläufe und die Nachvollziehbarkeit der Aufgaben geachtet.

Was ich gelernt habe

Mein wichtigster Lernschritt lag im Frontend und bei der SPA. Eine Seite ist nicht fertig, nur weil sie lokal richtig aussieht: Navigation, Rollenprüfung, API-Aufrufe und Deployment müssen zusammenpassen. Bei der SPA wurde das besonders deutlich. Die MPA sollte erhalten bleiben, während die neue Einstiegsseite Ansichten ohne vollständiges Neuladen wechselt. Dafür musste ich die Vorgaben aus ADR-08 und dem Sprint-Planning mit dem vorhandenen Frontend und den Schnittstellen abgleichen. Auch bei kleineren Responsive-Fixes habe ich gemerkt, dass eine Änderung bei 360 Pixeln gezielt geprüft werden muss und nicht automatisch aus der Desktopansicht folgt.

Beim Betrieb auf Heroku habe ich gelernt, wie stark Konfiguration und Reihenfolge den Erfolg eines Deployments beeinflussen. Frontend, core-service und marktdaten-service sind getrennte Apps; zugleich greifen die Backend-Dienste auf dieselbe Datenbank zu. Damit wurde für mich praktisch sichtbar, dass ein erfolgreicher Build allein noch nicht belegt, dass die Anwendung nach dem Start funktioniert. Zu einem Deploy gehören auch die Prüfung der Config Vars, der Build-Logs und der Health-Endpunkte. Die Apps und ihre Bereitstellung weitgehend selbst zu betreuen, hat mir einen besseren Blick für das Gesamtsystem gegeben.

Rückschläge und Zusammenarbeit

Ein wiederkehrendes Problem war, dass einzelne Issues nur einen Titel, aber keine Beschreibung enthielten. Bei größeren Frontend-Aufgaben musste ich deshalb Anforderungen aus Sprint-Planning-Protokollen, ADRs und Testdokumenten zusammentragen, bevor ich sicher wusste, was umgesetzt werden sollte. Das kostete Zeit und machte deutlich, dass eine gute technische Lösung mit einer klaren Aufgabenbeschreibung beginnt. Für künftige Sprints würde ich Akzeptanzkriterien und Abhängigkeiten direkt im Issue festhalten und früh prüfen, ob die beteiligten Repositories denselben Stand haben.

Auch die Arbeit mit Branches und Pull Requests hat mir gezeigt, wie wichtig eine saubere Trennung der Aufgaben ist. Änderungen an Navigation oder Routing können mehrere Seiten betreffen; deshalb helfen kleine, issuebezogene Branches und ein Abgleich mit `main`, bevor ein Fix weitergegeben wird. Beim Heroku-Betrieb gilt Ähnliches: Änderungen müssen mit den Merges im Frontend und Backend abgestimmt werden, damit ein Nachtest wirklich den erwarteten Stand prüft.

Arbeiten mit KI

KI-Unterstützung hat mir beim Einordnen von Anforderungen, beim Entwurf von Änderungen und beim Prüfen möglicher Ursachen geholfen. Gleichzeitig habe ich gelernt, Vorschläge nicht als Anforderung zu behandeln. Gerade bei leeren Issues kann ein Modell fehlenden Kontext plausibel ergänzen, obwohl die tatsächliche Vorgabe in einem anderen Dokument steht. Deshalb habe ich Ergebnisse mit den Projektunterlagen, dem vorhandenen Code und den Abnahmetestfällen abgeglichen. Für mich ist KI damit ein hilfreiches Werkzeug, wenn ich die Verantwortung für Prüfung und Entscheidung selbst behalte.

Bewertung des Umfangs

Rückblickend war der Umfang für die kurze Bearbeitungszeit sehr groß und zeitweise stressig. Neben den Aufgaben aus der Fallstudie kamen mit der SPA, dem Dashboard, den Responsive-Anforderungen und der Bereitstellung auf Heroku weitere technische Themen hinzu. Gerade im Frontend mussten Navigation, Rollen, API-Aufrufe und bestehende MPA-Seiten zusammenpassen. Parallel dazu liefen Abstimmung, Reviews und Dokumentation weiter. Ich hätte den Aufwand dafür zu Beginn stärker unterschätzt.

Ohne KI-Unterstützung wäre es für mich deutlich schwieriger gewesen, die vielen Anforderungen in der verfügbaren Zeit zu verstehen und umzusetzen. Sie half mir, technische Zusammenhänge schneller zu erschließen, Lösungswege zu vergleichen und bei Fehlern gezielter weiterzusuchen. Trotzdem musste ich die Vorschläge mit dem Code, den ADRs und den Abnahmekriterien abgleichen; gerade bei unvollständigen Issues durfte ich fehlende Vorgaben nicht einfach annehmen. Rückblickend waren die Kombination aus KI als Unterstützung, kleinen Arbeitsschritten und der Prüfung am laufenden System entscheidend, um die Aufgaben im gesetzten Zeitrahmen zu bewältigen.

5 Literatur

Zitiert wird im Text als (Autor Jahr) bzw. (Kurzname Jahr). Projektinterne Quellen (ADRs, API-Design, Protokolle, KI-Logs) werden mit Dateiname oder Issue-Nummer im Docs-Repository angegeben und hier nicht wiederholt.

- **Fielding 2000:** Fielding, R. T. (2000): *Architectural Styles and the Design of Network-based Software Architectures.* Dissertation, University of California, Irvine. Kapitel 5: Representational State Transfer (REST).
- **GraphQL Specification 2021:** GraphQL Foundation (2021): *GraphQL Specification, October 2021 Edition.* https://spec.graphql.org/October2021/
- **MACH Alliance:** MACH Alliance: *MACH: Microservices-based, API-first, Cloud-native SaaS and Headless.* https://machalliance.org
- **PostgreSQL Documentation:** PostgreSQL Global Development Group: *PostgreSQL Documentation, SELECT, The Locking Clause* (FOR UPDATE, SKIP LOCKED). https://www.postgresql.org/docs/current/sql-select.html
- **RFC 6750:** Jones, M.; Hardt, D. (2012): *The OAuth 2.0 Authorization Framework: Bearer Token Usage.* RFC 6750, IETF.
- **RFC 9110:** Fielding, R.; Nottingham, M.; Reschke, J. (2022): *HTTP Semantics.* RFC 9110, IETF.
- **Spring Framework Documentation:** Spring: *Data Access, Transaction Management, Transaction Propagation.* https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html
- **Twelve Data 2026:** Twelve Data: *Are the prices adjusted?* Support-Artikel. https://support.twelvedata.com/en/articles/5179064-are-the-prices-adjusted (abgerufen am 01.11.2026)